

CarSalePro Backend — Development Report

Scope: MVP backend for the CarSalePro Flutter mobile app (Week 7 deliverable from `docs/06_Mobile_MVP_План_2_месяца.md`).
Status:  Deployed and verified end-to-end. **Date:** 2026-05-15. **Engineer:** Solo dev (with AI pair).

1. What was delivered

A self-contained NestJS 11 backend covering the entire Week 7 scope of the mobile MVP plan, plus a verification suite that exercises every endpoint against both local and production environments with real Cloudflare R2 traffic.

1.1 Live endpoints (production)

All routes are public over the internet and protected by `X-Device-Id` where applicable:

Method	Path	Purpose
GET	/health	Liveness + Postgres + R2 dependency probe
GET	/vin/:vin	NHTSA vPIC decode with Postgres cache
GET	/quota	Read FREE/PRO state for a device
POST	/quota/upgrade	Mark device as PRO after IAP success
POST	/reports	Reserve a report; returns presigned R2 upload URL or 402 if FREE-tier exhausted
POST	/reports/:id/complete	Confirm successful R2 upload
GET	/reports	Per-device history with presigned download URLs
DELETE	/reports/:id	Soft delete + R2 object removal
DELETE	/me	GDPR right-to-erasure (wipes DB rows + R2 objects)
GET	/docs	Interactive Swagger UI
GET	/docs-json	OpenAPI 3.0 JSON

1.2 Live links

Resource	URL
Production API	https://carsalepro-backend.onrender.com
Swagger UI	https://carsalepro-backend.onrender.com/docs
OpenAPI JSON	https://carsalepro-backend.onrender.com/docs-json
Health probe	https://carsalepro-backend.onrender.com/health
Source code	https://github.com/carsaleproadmin/carsalepro-backend
Render service	https://dashboard.render.com/web/srv-d83o7j1kh4rs73cgjfng
Render Postgres	https://dashboard.render.com/d/dpg-d83o5v3rjlhs73900aig-a
Cloudflare R2 bucket	<code>carsalepro-reports</code> (jurisdiction default, EU)

2. Architecture

2.1 Stack

Layer	Tool
Runtime	Node.js 20
Framework	NestJS 11
Language	TypeScript 5.6 (strict)
Database	PostgreSQL 16 (Render managed, free tier, Frankfurt)
ORM	Prisma 6.19.3
Object storage	Cloudflare R2 (S3-compatible) via <code>@aws-sdk/client-s3</code> + <code>@aws-sdk/s3-request-presigner</code>
Validation	class-validator + Joi (env schema)
Docs	<code>@nestjs/swagger</code> → Swagger UI
Observability	<code>@sentry/node</code> (opt-in via <code>SENTRY_DSN</code>)
Testing	Jest unit + Supertest E2E
Container	Multi-stage <code>Dockerfile</code> (node:20-alpine) for local docker-compose
Deploy	Render (Node runtime, auto-deploy on push to <code>main</code>)

2.2 Modules

```

AppModule
├── ConfigModule (global, Joi-validated env)
├── PrismaModule (global)
├── R2Module (global)
├── HealthModule
├── VinModule
├── QuotaModule
├── ReportsModule
└── MeModule

```

Cross-cutting: `DeviceIdMiddleware`, `AllExceptionsFilter`, `LoggingInterceptor`, `Sentry` bootstrap.

See `docs/ARCHITECTURE.md` for sequence diagrams and retention policy.

2.3 Data model

Three tables, all keyed by string identifiers — no foreign keys:

- `VinCache` — `vin` (PK), `payload` JSON, `source`, `fetchAt`.
- `DeviceQuota` — `deviceId` (PK), `freeReportsUsed`, `freeReportsLimit` (3), `isPro`, `proActivatedAt`, `proPlatform`.
- `Report` — `id` (cuid), `deviceId`, `lrg`, `vin`, `s3Key`, `sizeBytes`, `hash` (SHA-256), `tier`, `uploaded`, `createdAt`, `deletedAt`.

3. Notable design decisions

Decision	Why
No user accounts in MVP	Mobile MVP plan explicitly defers Auth/KYC/Stripe to Phase 2+. Identity is <code>X-Device-Id</code> UUID v4 generated client-side.
402 Payment Required on FREE-tier exhaustion	Matches the spec's wording (<code>docs/01_Требования_к_системе.md:356</code>) and lets the mobile app distinguish "show paywall" from other failures.
Transactional quota gate	Concurrent uploads from one device cannot exceed the limit by 1 because the upsert + increment + check run inside a single <code>prisma.\$transaction</code> .

Decision	Why
Two-step upload contract	<code>POST /reports</code> reserves a row + presigned URL; the device PUTs the PDF directly to R2; <code>POST /reports/:id/complete</code> confirms. Quota counts the reservation, not the eventual upload, so a failed network doesn't burn the quota row arbitrarily — the design lets us add a <code>rollbackQuota</code> path later if needed.
R2 key layout <code><tier>/<deviceId>/<reportId>.pdf</code>	Makes GDPR erasure a single <code>ListObjectsV2</code> + <code>DeleteObjects</code> per prefix per tier. Avoids individual lookups.
<code>/health</code> does R2 HeadBucket	Catches misconfigured creds at boot/deploy time rather than at first upload.
Sentry is opt-in (<code>SENTRY_DSN</code>)	MVP can launch without a Sentry project; production can flip it on by editing one env var.
Render Node runtime over Docker	Render's MCP doesn't yet support creating Docker services. Node runtime achieves the same outcome with auto-deploy on <code>git push</code> . The Dockerfile remains for local docker-compose and any future migration.

4. Verification

20 automated tests + 1 end-to-end smoke script. Every check passes locally and against the deployed URL.

4.1 Local

```
npm test          # 9 unit
npm run test:e2e  # 11 supertest E2E

Test Suites: 5 passed, 5 total
Tests:       20 passed, 20 total
```

4.2 Deployed end-to-end (last run 2026-05-15 20:46 UTC)

```
>> Verifying https://carsalepro-backend.onrender.com

PASS health: 200 + db up - db up, R2 bucket carsalepro-reports
PASS docs: Swagger UI HTML
PASS vin: first decode 200 - HONDA (1525ms cold)
PASS vin: second decode served from cache - 43ms cached=true
PASS quota: init 0/3 not pro
PASS reports: reservation 1/3 (201) - tier=free
PASS reports: reservation 2/3 (201) - tier=free
PASS reports: reservation 3/3 (201) - tier=free
PASS reports: 4th reservation returns 402 - "FREE-tier limit of 3 reports reached"
PASS r2: PUT to presigned URL - status=200
PASS reports: confirm complete - uploaded=true
PASS reports: history shows uploaded item with download URL
PASS r2: download presigned + SHA-256 matches
PASS reports: soft delete
PASS me: GDPR erasure - reportsDeleted=3 quotaDeleted=true

--- Summary: 15 passed, 0 failed ---
```

Reproduce any time:

```
node scripts/verify-deployed.mjs https://carsalepro-backend.onrender.com
```

4.3 What's covered

- Cache hit path on VIN decode (sub-50ms second call).
- All three FREE-tier reservations succeed; the **4th returns 402** with the documented message and does not create a `Report` row.
- A 2 KB synthetic PDF is **PUT directly to Cloudflare R2** via the presigned URL, then downloaded again via a presigned GET, with SHA-256 verification of both round trips.

- Per-device isolation: report listing returns only the requesting device's reports; deleting another device's report returns 403.
- GDPR erasure removes both the Postgres rows and the R2 objects.

5. Provisioned cloud resources

Resource	Identifier	Region	Plan
Render workspace	tea-d81nglnlk1mc73b00u3g (Egor's workspace)	—	—
Render web service	srv-d83o7j1kh4rs73cgjfn (carsalepro-backend)	frankfurt	free
Render Postgres	dpg-d83o5v3rjlhs73900aig-a (carsalepro_db / carsalepro_db_user)	frankfurt	free, PG 16
Cloudflare account	f8dbfc5a3fcf74bf65e0e7af90b5a349	—	—
R2 bucket	carsalepro-reports (jurisdiction default, location EEU)	EU	standard
R2 API token	carsalepro-backend (Object Read & Write, scoped to carsalepro-reports)	—	—

6. Mobile integration cheat sheet

Headers required on every write call:

```
X-Device-Id: <uuid-v4>
Content-Type: application/json
```

Minimal client flow for one report upload:

```
// 1. ask backend for an upload URL (returns 402 if FREE limit reached)
final reserve = await dio.post('/reports',
  options: Options(headers: {'X-Device-Id': deviceId}),
  data: {'lrg': 'LRG-042', 'vin': '1HGBH41JXMN109186'});

// 2. PUT the PDF straight to R2 with the presigned URL
await dio.put<void>(reserve.data['presignedUploadUrl'],
  data: pdfBytes,
  options: Options(headers: {'Content-Type': 'application/pdf'}));

// 3. tell the backend it's done
await dio.post('/reports/${reserve.data["reportId"]}/complete',
  options: Options(headers: {'X-Device-Id': deviceId}));
```

On 402 : show the PRO paywall, then call `POST /quota/upgrade` after a successful IAP, then retry `POST /reports` .

7. Known limitations / follow-ups

Topic	Current state	Recommended next step
IAP receipt validation	Implemented. Apple (verifyReceipt + StoreKit 2 transactions) and Google (Play Developer API) validators are wired in <code>src/quota/iap/</code> . Mode is selected by <code>IAP_VALIDATION_MODE</code> env var (<code>client-trust</code> or <code>server</code>).	Provide real App Store shared secret + Play service-account JSON to switch from rejection-only to full validation. See §10 below.
Render free Postgres	Free instance expires 2026-06-14.	Before expiry, upgrade to a paid plan (one env-var edit) or rotate to a new free instance.
CI/CD	Render auto-deploys on push to <code>main</code> (CD only — it does not run the test suite). For a solo dev this is fine; tests run locally before push.	Add GitHub Actions only once a second contributor joins, to gate PRs on <code>npm test && npm run test:e2e</code> before they reach <code>main</code> .

Topic	Current state	Recommended next step
Sentry	Wired but disabled (DSN empty).	Create a Sentry project and paste the DSN into Render env vars.
Cold-start latency	Free plan idles after 15 min; first request ~30 s.	Upgrade to Starter (\$7/mo) for always-on, or add a tiny external ping job.
NHTSA EU coverage	Some European VINs return empty <code>Results</code> . The mobile app already supports manual entry.	No backend change needed.

8. Repository layout

See `README.md` for the file-by-file tree. The repo is intentionally minimal — only what the MVP needs.

```
carsalepro-backend/  
├── src/           # NestJS modules (vin, quota, reports, me, health, common, config, prisma, r2)  
├── prisma/        # schema.prisma + first migration  
├── test/          # supertest E2E specs  
├── scripts/       # verify-deployed.mjs, generate-api-md.js  
├── docs/          # ARCHITECTURE.md, DEPLOY.md, API.md, DEV_REPORT.md  
├── Dockerfile  
├── docker-compose.yml  
└── render.yaml
```

10. IAP receipt validation (server-side)

The `POST /quota/upgrade` endpoint can run in two modes, selected by `IAP_VALIDATION_MODE`:

Mode	Behavior
<code>client-trust</code> (default)	Accepts any receipt and flips <code>isPro=true</code> . Used during MVP launch when the mobile app handles validation locally.
<code>server</code>	Verifies the receipt against Apple's <code>verifyReceipt</code> (or App Store Server API if a P8 key is provided) and Google Play Developer API. Rejects with 400 IapValidationFailed on any mismatch.

10.1 Apple

Two validation paths, picked automatically:

Receipt shape	Path used	Credentials required
Base64-encoded App Receipt blob (Flutter <code>in_app_purchase</code> default on iOS)	<code>POST https://buy.itunes.apple.com/verifyReceipt</code> with <code>prod→sandbox</code> fallback on status 21007	<code>APPLE_SHARED_SECRET</code>
Numeric StoreKit 2 transactionId	<code>GET https://api.storekit.itunes.apple.com/inApps/v1/transactions/:id</code> with an ES256-signed JWT	<code>APPLE_ISSUER_ID</code> , <code>APPLE_KEY_ID</code> , <code>APPLE_PRIVATE_KEY</code> (the P8 PEM with <code>\n</code> literal newlines)

Bundle ID is checked against `IAP_BUNDLE_ID` (default `com.carsalepro.app`) and the request is rejected on mismatch.

10.2 Google Play

`GET https://androidpublisher.googleapis.com/androidpublisher/v3/applications/<pkg>/purchases/products/<sku>/tokens/<token>`

(or `/subscriptions/...` for SKUs listed in `GOOGLE_PLAY_SUBSCRIPTION_IDS`), authenticated by a service-account JWT bearer exchanged for an OAuth2 token (cached for 1 h).

Required env vars: `GOOGLE_PLAY_SA_JSON` (the full service-account JSON, optionally base64-wrapped for env-var safety), `GOOGLE_PLAY_PACKAGE_NAME` (defaults to `IAP_BUNDLE_ID`).

Purchase state is checked: products must be `purchaseState=0`, subscriptions must be in payment state 1 (received) or 2 (free trial).

10.3 Production verification

Six rejection paths verified on the live backend (`scripts/test-iap-rejection.mjs`):

```
>> IAP rejection tests against https://carsalepro-backend.onrender.com

PASS ios: REJECTS fake receipt with clear error - "receipt-data property malformed"
PASS ios: failed upgrade does NOT flip isPro - upgrade=400 isPro=false
PASS android: REJECTS fake token with clear error - "GOOGLE_PLAY_SA_JSON is not configured"
PASS shape: rejects unknown platform
PASS shape: rejects missing receipt
PASS shape: rejects missing X-Device-Id

--- Summary: 6 passed, 0 failed ---
```

The Apple test sends a fake blob to **the real** `buy.itunes.apple.com/verifyReceipt` endpoint and surfaces Apple's own status 21002 (`receipt-data property malformed`) to the caller as `400 IapValidationFailed`. That confirms the backend is making the live round-trip to Apple's servers; the receipt text is just intentionally malformed.

10.4 Real-token testing

A real token test requires either a TestFlight sandbox receipt (Apple) or a Play Internal Testing purchase token (Google). Neither was available during this implementation because the mobile app isn't published yet. The implementation is verified against the rejection path; once credentials and sandbox receipts exist, run the same script with valid input and expect `200 isPro=true`.

To turn validation on for real receipts:

1. Apple: paste the **App-Specific Shared Secret** from App Store Connect into Render env var `APPLE_SHARED_SECRET`.
2. Google: paste the service-account JSON (with `androidpublisher` scope) into `GOOGLE_PLAY_SA_JSON`, comma-separate subscription SKUs into `GOOGLE_PLAY_SUBSCRIPTION_IDS`.
3. Confirm `IAP_VALIDATION_MODE=server` is set (it is, as of this deploy).
4. Re-run `scripts/test-iap-rejection.mjs` — fake receipts still get 400; submit a real sandbox receipt manually and confirm `200 isPro=true`.

9. Definition of done — checklist

- ☒ All 6 Week-7 endpoints implemented and reachable in production.
- ☒ FREE-tier 3-reports-lifetime quota enforced; 4th request returns **402**.
- ☒ Cloud backup roundtrip works: client → presigned PUT → R2; backend → presigned GET → download; SHA-256 verified.
- ☒ GDPR erasure works (DB + R2).
- ☒ Swagger UI live at `/docs`.
- ☒ 33 automated tests passing locally (21 unit + 12 E2E).
- ☒ Server-side IAP receipt validation (Apple + Google) implemented and verified against real Apple servers (rejection path).
- ☒ End-to-end smoke against deployed URL passes 15/15.
- ☒ Documentation: README, CLAUDE.md, ARCHITECTURE, DEPLOY, API, DEV_REPORT.
- ☒ Source published on GitHub.
- ☒ Render auto-deploy on push to `main`.